

Chapter 1 – Exercise Solve

1.1 What are the three main purposes of an operating system?

Operating System (OS) এর ৩টা main purpose খুব সহজভাবে বুঝো 🙌

1 Program run করার environment দেওয়া

OS এমন একটা পরিবেশ তৈরি করে যাতে আমরা সহজে program চালাতে পারি।

🙌 যেমন: তুমি যখন Windows এ VS Code বা Chrome চালাও, এটা OS এর কারণেই সম্ভব।
মানে OS না থাকলে software চলতো না।

2 Computer resource manage করা

Computer এর CPU, RAM, Hard Disk, I/O device — এগুলো limited।
OS ঠিক করে দেয় কে কতটুকু ব্যবহার করবে।

🙌 ধরো একসাথে Chrome, VS Code, আর Music player চালাচ্ছে —
OS ঠিকভাবে RAM ও CPU ভাগ করে দেয় যাতে সব ঠিকমতো চলে।
এটা যেন ন্যায্য (fair) ও efficient হয়।

3 Control Program হিসেবে কাজ করা

OS দুইটা গুরুত্বপূর্ণ কাজ করে:

- ◆ Error prevent করা – কোনো program যেন ভুলভাবে system নষ্ট না করে।
 - ◆ I/O device manage করা – যেমন keyboard, mouse, printer, disk ইত্যাদি ঠিকভাবে কাজ করানো।
-

1.2 We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?

এটা খুব সহজভাবে বুঝো 🙌

আমরা বলি OS এর কাজ হলো hardware efficient ভাবে ব্যবহার করা।

কিন্তু কিছু সময় OS ইচ্ছা করে কিছু resource “waste” করে।

কখন এটা ঠিক?

👉 **Single-user system** (যেমন তোমার personal laptop)।

এখানে main লক্ষ্য হলো **user experience ভালো করা**, শুধু CPU বাঁচানো না।

উদাহরণ:

তুমি যখন **Graphical User Interface (GUI)** ব্যবহার করো (Windows, icons, animations ইত্যাদি),

এগুলো extra CPU ও RAM ব্যবহার করে।

মানে technically একটু “waste” হচ্ছে।

তাহলে এটা waste না কেন?

কারণ:

✓ User এর কাজ সহজ হয়

✓ Interaction দ্রুত ও comfortable হয়

✓ Productivity বাড়ে

অর্থাৎ CPU একটু বেশি ব্যবহার হলেও,

User এর সুবিধা বাড়ছে — তাই এটা আসলে waste না।

এক লাইনে উত্তর:

যখন user experience improve করা বেশি গুরুত্বপূর্ণ (বিশেষ করে single-user system এ), তখন OS কিছু resource বেশি ব্যবহার করতে পারে। এটা আসলে waste না, কারণ এতে user এর কাজ দ্রুত ও সহজ হয়।

1.3 What is the main difficulty that a programmer must overcome in writing an operating system for a real-time environment?

এটা খুব সহজভাবে বুঝো 👉

● Real-Time System মানে কী?

Real-time system এ কাজ **নির্দিষ্ট সময়ের মধ্যে শেষ করতেই হবে**।

সময় মিস করলে বড় সমস্যা হতে পারে।

👉 যেমন:

- Airplane control system
- Medical equipment
- Industrial machine

🔥 Main Difficulty কী?

Operating System বানানোর সময় সবচেয়ে বড় সমস্যা হলো:

🕒 Fixed time limit এর মধ্যে সব কাজ শেষ করা

যদি নির্দিষ্ট সময়ের মধ্যে কাজ শেষ না হয়,
তাহলে পুরো system crash বা breakdown হতে পারে।

🧠 Programmer এর চ্যালেঞ্জ

- ✓ Scheduling এমনভাবে করতে হবে যাতে delay না হয়
- ✓ Response time কখনো time limit ছাড়িয়ে না যায়
- ✓ System সবসময় predictable হতে হবে

সহজ কথায় উত্তর:

Real-time OS বানানোর সবচেয়ে বড় সমস্যা হলো নির্দিষ্ট সময়ের মধ্যে কাজ শেষ নিশ্চিত করা।
সময় মিস হলে পুরো system নষ্ট হতে পারে।

1.4. Keeping in mind the various definitions of operating system, consider whether the operating system should include applications such as web browsers and mail programs. Argue both that it should and that it should not, and support your answers.

এটা প্রশ্নটা বলছে —

Web browser, Mail program এর মতো application কি Operating System এর ভিতরে রাখা উচিত?

চলো দুই দিক থেকেই সহজভাবে বুঝি 🙌

✅ কেন OS এর ভিতরে রাখা উচিত (Yes, should include)

1 Better Performance

যদি browser বা mail program OS এর ভেতরে (kernel এর কাছাকাছি) থাকে, তাহলে সেটা system feature ভালোভাবে ব্যবহার করতে পারবে।

👉 ফলে speed ও performance বাড়তে পারে।

2 Better Integration

OS এর সাথে tightly connected থাকলে app গুলো আরও smoothly কাজ করবে।

✗ কেন রাখা উচিত না (No, should NOT include)

এই দিকটাই বেশি strong 📢

1 Application আর OS এক জিনিস না

Browser, mail program — এগুলো application।

OS এর কাজ হলো system manage করা, app বানানো না।

2 Security Risk বাড়ে

Kernel এর ভিতরে app থাকলে bug হলে পুরো system crash হতে পারে।

Security vulnerability বাড়ে।

3 OS অনেক বড় (bloated) হয়ে যায়

অপ্রয়োজনীয় app ঢুকালে OS heavy হয়ে যায়।

সব user তো একই app ব্যবহার করে না।

🎯 Final Conclusion (Exam এর জন্য)

দুই দিকেই যুক্তি আছে,

কিন্তু সাধারণভাবে application গুলো OS এর বাইরে থাকাই ভালো,

কারণ এতে security বেশি থাকে এবং OS lightweight থাকে।

এক লাইনে:

Performance বাড়ার জন্য OS এর ভিতরে রাখা যেতে পারে,

কিন্তু security ও design principle এর জন্য বাইরে রাখা বেশি ভালো।

1.5 How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security)?

◆ 1.5 Kernel Mode আর User Mode কীভাবে security দেয়?

Computer এ দুই ধরনের mode থাকে:

● User Mode

এখানে সাধারণ program (Chrome, VS Code ইত্যাদি) চলে।

এরা system এর sensitive জিনিস সরাসরি access করতে পারে না।

● Kernel Mode

এখানে Operating System চলে।

এটা পুরো system control করতে পারে —

CPU, Memory, I/O device সব।

🔒 তাহলে Security কিভাবে হয়?

✓ কিছু instruction শুধু kernel mode এ চালানো যায়

✓ Hardware device শুধু kernel mode থেকেই access করা যায়

✓ Interrupt on/off শুধু kernel mode এ করা যায়

👉 তাই user program ইচ্ছা করলে system নষ্ট করতে পারে না।

সহজ কথায়:

User mode এ ক্ষমতা কম দেওয়া হয়,

Kernel mode এ পুরো control থাকে।

এভাবেই system protected থাকে।

1.6 Which of the following instructions should be privileged?

a. Set value of timer.

b. Read the clock.

c. Clear memory.

d. Issue a trap instruction.

e. Turn off interrupts.

f. Modify entries in device-status table.

g. Switch from user to kernel mode.

h. Access I/O device.

◆ 1.6 কোন instruction গুলো privileged হওয়া উচিত?

👉 Privileged মানে শুধু kernel mode এ করা যাবে।

✅ Privileged Instruction:

a. ✓ Set value of timer

c. ✓ Clear memory

e. ✓ Turn off interrupts

f. ✓ Modify device-status table

h. ✓ Access I/O device

কারণ এগুলো system control করে —

ভুল করলে পুরো system crash হতে পারে।

❌ Privileged না (User mode এ করা যায়):

b. ✗ Read the clock

d. ✗ Issue a trap instruction

g. ✗ Switch from user to kernel mode

🔧 এক লাইনে মনে রাখো:

যেগুলো system control বা hardware directly affect করে,

সেগুলো privileged হবে।

1.7 Some early computers protected the operating system by placing it in a memory partition that could not be modified by either the user job or the operating system itself. Describe two difficulties that you think could arise with such a scheme.

◆ 1.7 OS কে আলাদা memory partition এ রাখলে কী সমস্যা হতে পারে?

ধরো OS এমন এক জায়গায় রাখা হলো যেটা কেউ modify করতে পারবে না —
না user, না OS নিজেও।

🚨 সমস্যা ১: গুরুত্বপূর্ণ data unprotected হয়ে যাবে

OS এর দরকারি data যেমন:

- password
- access control
- accounting info

এসব যদি protected memory তে রাখা না যায়,
তাহলে সেগুলো normal memory তে রাখতে হবে।

👉 এতে unauthorized user এগুলো দেখতে বা বদলাতে পারে।

🚨 সমস্যা ২: OS update বা পরিবর্তন করা কঠিন

যেহেতু OS নিজেও modify করতে পারছে না,
তাই bug fix, update, configuration change করা কঠিন হয়ে যাবে।

🌀 সহজ কথা:

OS পুরোপুরি locked করে রাখলে security কিছু বাড়ে,
কিন্তু flexibility আর data protection কমে যায়।

1.8 Some CPUs provide for more than two modes of operation. What are two possible uses of these multiple modes?

◆ 1.8 দুইটার বেশি mode থাকলে কী কাজে লাগতে পারে?

সাধারণত ২টা mode থাকে: User & Kernel
কিন্তু কিছু CPU তে ৩ বা তার বেশি mode থাকে।

● ব্যবহার ১: Fine-grained security

সব user এক রকম না।
ধরো একই group এর user একে অপরের program চালাতে পারবে।

👉 তাহলে আলাদা group-mode রাখা যায়।

● ব্যবহার ২: Kernel এর ভিতরেও আলাদা level

ধরো USB driver এর জন্য আলাদা mode দিলাম।

তাহলে পুরো kernel mode এ না গিয়ে

একটা specific mode এ driver চলতে পারবে।

👉 এতে efficiency বাড়ে এবং security maintain থাকে।

🌀 সহজ কথা:

Multiple mode মানে বেশি control + বেশি security level।

1.9 Timers could be used to compute the current time. Provide a short description of how this could be accomplished.

◆ 1.9 Timer দিয়ে current time কিভাবে বের করা যায়?

Timer interrupt ব্যবহার করে সময় হিসাব করা যায়।

কীভাবে?

- 1 Program একটা timer set করবে (ধরো ১ সেকেন্ড পর interrupt হবে)
 - 2 Program sleep করবে
 - 3 Timer interrupt হলে program জেগে উঠবে
 - 4 Interrupt কয়বার হয়েছে সেটা count করবে
 - 5 এই count থেকে total time হিসাব করবে
-

🌀 সহজ উদাহরণ:

যদি প্রতি ১ সেকেন্ডে interrupt হয়

আর ৬০ বার interrupt হয়েছে

👉 তাহলে ৬০ সেকেন্ড = ১ মিনিট

এক লাইনে:

Timer interrupt গুনে গুনে program নিজেই সময় হিসাব করতে পারে।

1.10 Give two reasons why caches are useful. What problems do they solve? What problems do they cause? If a cache can be made as large as the device for which it is caching (for instance, a cache as large as a disk), why not make it that large and eliminate the device?

◆ **Cache কেন দরকার? (Two reasons)**

1 Speed difference সমস্যা সমাধান করে

ধরো CPU খুব fast, কিন্তু RAM বা Disk ধীর।

তাহলে CPU কে বারবার slow device এর জন্য অপেক্ষা করতে হবে।

👉 Cache মাঝখানে একটা **দ্রুত buffer** হিসেবে কাজ করে।

CPU আগে cache এ খোঁজে।

পেয়ে গেলে slow device এ যেতে হয় না।

2 Performance বাড়ায়

বারবার ব্যবহার হওয়া data cache এ থাকলে

system অনেক দ্রুত কাজ করে।

👉 তাই overall efficiency বেড়ে যায়।

◆ **Cache কী সমস্যা সমাধান করে?**

✓ Fast আর slow device এর মধ্যে speed gap কমায়

✓ Data transfer দ্রুত করে

✓ CPU idle সময় কমায়

◆ **Cache কী সমস্যা তৈরি করে?**

⚠ **1. Consistency problem**

যদি main memory তে data বদলায়,

তাহলে cache এর data ও update করতে হবে।

না হলে দুই জায়গায় আলাদা data থাকবে।

👉 এটাকে বলে **cache consistency problem**

Multiprocessor system এ এটা বড় সমস্যা।

⚠️ 2. Extra complexity

Cache manage করা জটিল।

Hardware ও OS কে sync রাখতে হয়।

◆ **Cache যদি disk এর মতো বড় বানানো যায়, তাহলে disk বাদ দিই না কেন?**

ভালো প্রশ্ন 👉

দুইটা শর্ত লাগবে:

1 Power গেলে data থাকতে হবে

Disk power off হলেও data রাখে (non-volatile)।

কিন্তু cache (RAM type) সাধারণত power গেলে data হারায়।

👉 তাই same storage capacity না হলে replace করা যাবে না।

2 Cost অনেক বেশি

Fast memory (cache) খুব expensive।

Disk অনেক সস্তা।

👉 তাই পুরো disk size এর cache বানানো practical না।

🎯 Final Short Answer (Exam এর জন্য)

Cache useful কারণ এটি fast ও slow device এর মধ্যে speed gap কমায় এবং performance বাড়ায়।

কিন্তু consistency ও cost সমস্যা তৈরি করে।

Disk এর সমান বড় cache বানানো সম্ভব হলেও power retention ও cost এর কারণে disk replace করা যায় না।

1.11 Distinguish between the client-server and peer-to-peer models of dis-tributed systems.

◆ Client-Server vs Peer-to-Peer (P2P)

1 Client-Server Model

- **Roles fixed:** Client & Server আলাদা।
- **Client:** service request করে
- **Server:** service provide করে
- উদাহরণ: Web browsing
 - তুমি browser (client) এ website চাও
 - Web server সেটা send করে

Key point: Server সব service control করে, client শুধু request করে।

2 Peer-to-Peer (P2P) Model

- **No fixed roles:** সব node peer
- **Node:** চাইলে service request করতে পারে, চাইলে provide করতে পারে
- উদাহরণ: Torrent system বা recipe sharing
 - Node A recipe চায় → Node B বা C দিয়ে দেয়
 - Node B ও আবার অন্য peer কে service দিতে পারে

Key point: সবাই client & server দুটোই হতে পারে।

◆ সহজ কথায় তুলনা

Feature	Client-Server	Peer-to-Peer
Role	Fixed (Client / Server)	Flexible (Peer = client & server)
Control	Server controls all	No central control

Feature	Client-Server	Peer-to-Peer
Example	Web server	Torrent, File sharing

🔗 মূল কথা:

- Client-server → Central server, roles fixed
- P2P → Decentralized, flexible roles, সবাই একে অপরকে serve করতে পারে

Chapter 2 – Exercise Solve

2.1 What is the purpose of system calls?

♦ 2.1 Purpose of System Calls

System calls হলো user program এর জন্য OS এর সেবা চাওয়ার মাধ্যম।

- User-level process direct hardware access করতে পারে না।
- তাই system call করে OS কে বলে:
👉 “আমি এই কাজ করতে চাই, তুমি করে দাও।”

উদাহরণ: File open করা, memory allocate করা, print command দেওয়া।

2.2 What is the purpose of the command interpreter? Why is it usually separate from the kernel?

♦ 2.2 Purpose of Command Interpreter (Shell)

Command interpreter / shell এর কাজ:

- 1 User বা command file থেকে **commands** পড়া
- 2 Commands execute করা
- 3 সাধারণত system calls ব্যবহার করে কাজ করা

Separate কেন kernel থেকে?

- Shell অনেক পরিবর্তনযোগ্য (update/change) হতে পারে
- Kernel খুব sensitive, বারবার update করা ঝুঁকিপূর্ণ

- তাই shell আলাদা রাখা হয়

2.3 What system calls have to be executed by a command interpreter or shell in order to start a new process on a UNIX system?

◆ 2.3 System Calls to Start a New Process (UNIX)

UNIX এ process start করতে দুটি main system call লাগে:

- 1 **fork()** → বর্তমানে চলা process কে copy করে নতুন process বানায়
- 2 **exec()** → নতুন executable program কে overlay করে, নতুন process চালায়

Flow:

- Shell → fork() → exec() → নতুন process শুরু

2.4 What is the purpose of system programs?

◆ 2.4 Purpose of System Programs

System programs হলো pre-made programs যা common tasks সহজ করে।

- User নিজে সব program লিখতে হয় না
- OS এর system calls এর উপর base করে বানানো

উদাহরণ:

- File management (cp, mv, rm)
- Text editors (vi, nano)
- Compilers, shells

Short summary:

System programs = user-friendly packages of system calls.

2.5 What is the main advantage of the layered approach to system design? What are the disadvantages of the layered approach?

◆ Layered Approach to System Design

Layered approach:

Operating system কে বিভিন্ন layer বা module এ ভাগ করা হয়।

- প্রতিটি layer শুধু তার নীচের layer ব্যবহার করে

- উপরের layer শুধু তার নিচের layer থেকে service নেয়
-

✓ Main Advantage (ফায়দা)

Modular design এর সুবিধা:

1 Debug & Modify সহজ হয়

- সমস্যা হলে শুধু ওই layer দেখলেই হয়
- পুরো system change করতে হয় না

2 Data safety & isolation

- প্রতিটি layer শুধু নিজের data access করে
 - Bug impact কমে, এক layer এর bug অন্য layer এ ছড়ায় না
-

✗ Disadvantages (অসুবিধা)

1 Performance কমে

- Service পেতে অনেক layer traverse করতে হয়
 - Layer traversal → extra overhead → system ধীর হতে পারে
-

🔗 এক লাইনে মনে রাখো:

Advantage: Easy to maintain, debug, modify, and isolated data

Disadvantage: Slower performance due to multiple layer traversal

2.6 List five services provided by an operating system, and explain how each creates convenience for users. In which cases would it be impossible for user-level programs to provide these services? Explain your answer.

♦ 5 Services Provided by an Operating System

1 Program Execution

কী করে: OS file থেকে program load করে memory তে, তারপর run করে।

User convenience: User কে CPU allocation বা program start করার জটিলতা দেখতে হয় না।

User-level program কেন পারে না:

- CPU time ঠিকভাবে allocate করা user program trust করা যায় না।
-

2 I/O Operations

কী করে: OS disk, tape, printer, etc. এর সাথে low-level communication handle করে।

User convenience: শুধু device select করে operation বলতে হয়, OS সেই request device-specific command এ convert করে দেয়।

User-level program কেন পারে না:

- User program ভুল device access করতে পারে বা device busy থাকলে conflict হতে পারে।
-

3 File-System Manipulation

কী করে: File creation, deletion, allocation, naming, protection, free/used block tracking সব manage করে।

User convenience: User কে disk blocks, file protection, allocation detail নিয়ে মাথা ঘামাতে হয় না।

User-level program কেন পারে না:

- Protection enforce করতে পারে না
 - Free/allocated block ঠিকভাবে manage করতে পারে না
-

4 Communications

কী করে: OS messages কে packets এ convert করে, network transmit করে, destination এ reassemble করে।

User convenience: User শুধু message পাঠাবে, OS network handling করে।

User-level program কেন পারে না:

- Network coordination tricky

- অন্য process এর packets accidentally receive করতে পারে

5 Error Detection

কী করে:

- Hardware & software level এ data corruption detect করে
- Disk, memory, network errors check করে
- Processes কে error handling code লিখতে হয় না
User convenience: User বা program শুধু কাজ করে, OS automatically errors catch & correct করে।

User-level program কেন পারে না:

- Errors system-wide, multiple processes affect করে
- Global handling ছাড়া individual program সব error catch করতে পারবে না

 সহজ সংক্ষেপে:

Service	Convenience	Why user program can't do it
Program Execution	Program run করা সহজ	CPU allocation trustable নয়
I/O Operations	সহজ I/O	Device access safe নয়
File-System Manipulation	Disk management automatic	Protection & allocation ঠিক রাখতে পারবে না
Communications	Network handle করা সহজ	Coordination & packet control সমস্যা
Error Detection	Automatic error handling	System-wide errors program handle করতে পারবে না

2.7 Why do some systems store the operating system in firmware, while others store it on disk?

♦ 2.7 OS Firmware vs Disk এ কেন রাখা হয়?

1 OS in Firmware

কখন: যখন device এ disk বা file system নেই

উদাহরণ: Embedded systems, microcontrollers

কারণ: OS সরাসরি device এর memory তে থাকতে হবে, যাতে boot করা যায়।

2 OS on Disk

কখন: যখন disk available থাকে

উদাহরণ: Desktop, Laptop

কারণ: Disk-এ OS update করা সহজ, storage বেশি, flexible।

🔗 সহজ সংক্ষেপে:

- **Firmware:** Disk নেই, fixed OS, fast boot
- **Disk:** Update & multiple OS possible

2.8 How could a system be designed to allow a choice of operating systems from which to boot? What would the bootstrap program need to do?

♦ 2.8 Multiple OS Boot করার জন্য System Design

ধরি তোমার system এ Windows + 3 Linux distro (RedHat, Debian, Ubuntu) আছে।

কীভাবে করা যায়:

1 Boot Manager তৈরি করা

- System startup এ OS এর আগে এই program run হবে
- Decide করবে কোন OS boot করতে হবে

2 OS Disk এ রাখা

- প্রতিটি OS আলাদা location/partition এ থাকবে
- Boot manager সেই location থেকে OS load করবে

3 User selection

- Boot menu দেখাবে, user select করবে
- যদি select না করে → default OS boot হবে

◆ Bootstrap Program এর কাজ:

- Power-on → Bootstrap run
- First OS না, **Boot Manager run**
- Boot manager user input check করে OS load করবে

🌀 সহজে মনে রাখার জন্য:

Feature	Explanation
Firmware OS	Disk না থাকলে, embedded device এ, fixed OS
Disk OS	Updateable, multiple OS possible
Boot Manager	Decide which OS to load, show menu, load selected OS

Chapter 6 – Exercise Solve

6.1 In Section 6.4, we mentioned that disabling interrupts frequently can affect the system's clock. Explain why this can occur and how such effects can be minimized.

◆ 6.1 Why disabling interrupts affects the system clock

Key points:

1 Clock updates via interrupts

- System clock প্রতি **clock interrupt** এ update হয়।

2 Problem if interrupts disabled

- Interrupt দীর্ঘ সময়ের জন্য disable করলে
 - clock ticks miss হবে
 - system clock ভুল হয়ে যাবে

3 Impact on scheduling

- Process time quantum clock ticks দ্বারা measure হয়
- Interrupt disable হলে scheduler ঠিকভাবে time quantum assign করতে পারবে না

4 Solution / Minimization

- Interrupt শুধু খুব **short period** এর জন্য disable করতে হবে
- Long disable করলে timing & scheduling ঠিক হবে না

🔗 সহজে মনে রাখার shortcut:

“Interrupt off → clock ticks miss → wrong time & scheduling; minimize by short disable only”

6.2 What is the meaning of the term busy waiting? What other kinds of waiting are there in an operating system? Can busy waiting be avoided altogether? Explain your answer.

◆ 6.2 Busy Waiting

Meaning:

Busy waiting = process এক condition complete হওয়ার জন্য **tight loop** এ **wait** করা, কিন্তু processor ছাড়ে না।

```
while (condition not true) {  
    // do nothing  
}
```

Other kinds of waiting:

1 Blocking / Sleep waiting

- Process wait করে কিন্তু CPU free থাকে
 - Wake-up signal এ process resume হয়
-

Can busy waiting be avoided?

- **Partially yes:**
 - Sleep/wakeup strategy ব্যবহার করলে CPU release হয়
 - কিন্তু এতে **overhead** আসে (sleep & wake-up cost)
-

Shortcut:

Busy waiting = CPU খালি loop এ wait; Sleep waiting better, overhead trade-off থাকে

6.3 Explain why spinlocks are not appropriate for single-processor systems yet are often used in multiprocessor systems.

◆ 6.3 Spinlocks: Single vs Multiprocessor

Spinlock meaning:

- Process constantly check করে lock, CPU ছাড়ে না until lock available

Single-processor problem:

- Single CPU → only one process executes

- Lock holder release করতে অন্য process execute হতে হবে
- কিন্তু spinlock looping process CPU ছাড়ে না → **deadlock / wasted CPU**

Multiprocessor advantage:

- Other processors run other processes
- তারা lock release করতে পারে → first process spinlock ছাড়তে পারে
- তাই multiprocessor system এ spinlocks useful

Shortcut summary:

System	Spinlock suitable?	Reason
Single CPU	✗ No	Process holding lock can't be scheduled → deadlock
Multi CPU	✓ Yes	Other CPUs can release lock → progress possible

6.4 Show that, if the wait () and signal () semaphore operations are not executed atomically, then mutual exclusion may be violated.

♦ 6.4 Non-atomic wait() and signal() → Mutual Exclusion Violation

Semaphore recap:

- Semaphore = counter + wait()/signal() operations
- **Binary semaphore** (mutex) → 0 বা 1

Problem:

- wait() normally **atomically** decrement করে semaphore value
- যদি **non-atomic** হয় → multiple processes একসাথে access করতে পারে

Example:

- Semaphore value = 1
- Process A: wait() starts → read value = 1
- Process B: wait() starts → read value = 1

- Both decrement $\rightarrow$ value = -1
 - দুটো process critical section এ একসাথে ঢুকে যাবে $\rightarrow$ **mutual exclusion violated**
-

Shortcut:

wait() / signal() must be atomic, otherwise 2 process একসাথে critical section এ ঢুকতে পারে

6.5 Illustrate how a binary semaphore can be used to implement mutual exclusion among n processes.

◆ 6.5 Binary Semaphore for n-process Mutual Exclusion

Setup:

- Shared semaphore: mutex = 1
- n processes: P1, P2, ..., Pn

Pseudocode:

```
do {  
    wait(mutex);    // enter critical section  
  
    /* critical section */  
  
    signal(mutex);  // exit critical section  
  
    /* remainder section */  
} while (true);
```

How it works:

- Only 1 process enter critical section because semaphore = 1
- Others wait until mutex = 1 again
- Ensures **mutual exclusion among n processes**

6.6 Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: deposit (amount) and withdraw(amount). These two functions are passed the amount that is to be deposited or withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the withdraw() function, and the wife calls deposit (). Describe how a race condition is possible and what might be done to prevent the race condition from occurring.

◆ **6.6 Race Condition Example (Bank Account)**

Scenario:

- Account balance = \$250
- Husband: withdraw(\$50)
- Wife: deposit(\$100)

Steps (interleaving):

1. Husband reads balance → 250
2. Husband computes $250 - 50 = 200$
3. Before committing, Wife reads balance → 250
4. Wife computes $250 + 100 = 350$ → writes 350
5. Switch back to Husband → writes 200

Result: balance = 200 ❌ (Incorrect)

Solution to prevent race:

- **Mutual exclusion:** protect access to shared balance
- Use **binary semaphore / mutex:**

```
wait(mutex);
```

```
balance = balance - 50; // withdraw
```

```
signal(mutex);
```

```
wait(mutex);
```

balance = balance + 100; // deposit

signal(mutex);

- Only 1 process accesses balance at a time → race avoided

Shortcut Summary:

Concept	Problem	Solution
Non-atomic wait()	Multiple process enter CS → mutual exclusion violated	Make wait()/signal() atomic
Binary semaphore	n processes → mutual exclusion	wait(mutex)/signal(mutex) around CS
Race condition	Concurrent access → incorrect balance	Use mutex/semaphore around shared variable